

QA & BA Strategy

What this platform must do, how we intend to prove it does, what has already been proven, and the four defects found so far. Written for the HumAId Risk Governance team ahead of the 30 September submission.

AUTHOR	DATE	BRANCH	DAYS TO SUBMIT
Alagappan Annamalai · BA / QA	12 September 2026	qa/ba-qa-workstream	18

74	4	12	0
AUTOMATED TESTS	DEFECTS RAISED	AI EVAL CASES	CI PIPELINES RUNNING THEM

01 Requirement understanding & architecture

What we are building, for whom, and the shape of the system that delivers it.

The business problem

A national bank (~\$500B assets) must assess financial-crime risk before any business change goes live — a new product, a feature, a process change, a vendor, a new geography, or a new customer segment. Today that assessment runs on email, Word, Excel and SharePoint. It takes **15–20 business days**, two analysts can reach different conclusions on the same change, and reconstructing the reasoning behind a past rating for an examiner is slow and incomplete.

The Risk Assessment Workbench carries a change request through **intake** → **assessment** → **committee decision** on one governed platform, with an audit trail an examiner can read.

Actors

ACTOR	DOES	MUST NEVER BE ABLE TO
Product Owner	Raises the change request, attaches documents, tracks status	See analyst scoring rationale; act on the assessment itself
FCRM Analyst	Reviews AI output, overrides with reason, scores, finalizes, owns configuration	Route an unreviewed assessment to committee
Risk Committee	Reviews the finalized assessment and votes	Vote anonymously or have a vote aggregated away

The three constraints everything hangs off

- 1. **The system prepares, humans decide.** Nothing is auto-approved or auto-rejected. Every AI touchpoint pairs with a human review *gate in code*.
- 2. **Controls mitigate, never eliminate.** Residual risk can never reach zero — enforced at calculation time *and* at configuration time.
- 3. **Grounded in a citable framework.** Risk decomposition uses the FFIEC BSA/AML Examination Manual's four categories. Nothing invented, every category carries a citation.

WHY THIS MATTERS FOR TEST DESIGN

These three are not features — they are invariants. A test suite for this product is judged on whether it can *prove* them, not on line coverage. That is the organising principle for everything in sections 4 and 5.

The ten epics

#	EPIC	AI?	PAIRED HUMAN GATE
1	Change Request Intake	NO	—
2	Risk Categorization & Framework Mapping	AI	Analyst adds/removes category, reason mandatory
3	Policy Research	NO — Postgres FTS by design	Analyst marks each policy relied-upon / not relevant
4	Document Extraction	AI	Analyst corrects any field; low-confidence pre-flagged
5	AI-Drafted Risk Assessment	AI	Section cannot route to committee until reviewed/edited
6	Analyst Review, Edit & Override	NO	Is the gate — reason mandatory everywhere
7	Risk Scoring Engine	NO — fixed formula by design	Analyst override with reason; residual > 0 always
8	Committee Review & Voting	NO	Individual votes never aggregated anonymously
9	Immutable Audit Trail	NO	Append-only enforced by DB trigger
10	Platform Configuration	NO	Config change is itself an audit event

System architecture

A 5-layer Clean Architecture .NET solution using Dapper against named PostgreSQL stored functions — never an ORM, never ad-hoc SQL. Mock External Systems is deliberately a *separate* service, standing in for the CRM / Core Banking / Vendor Management systems a real bank already runs.

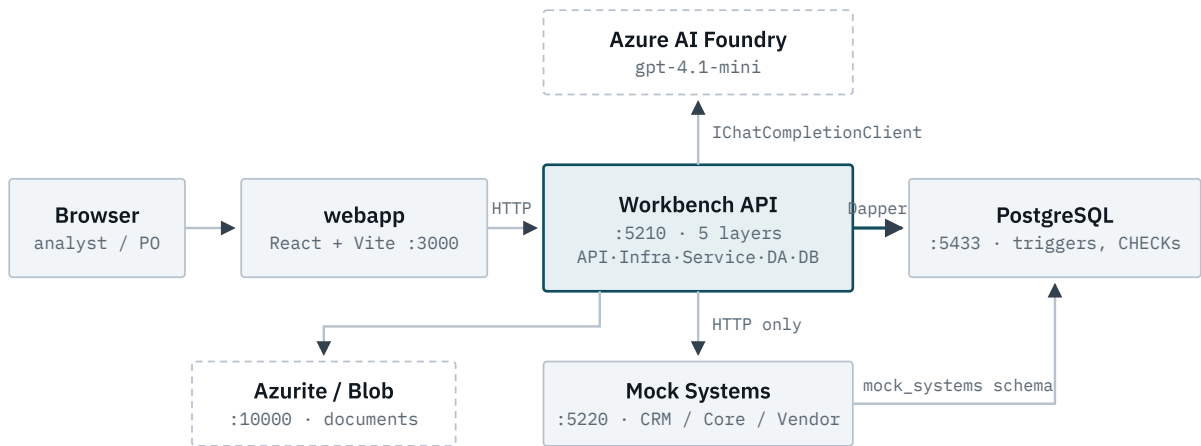


Figure 1. The Workbench is the only thing that talks to Mock Systems, and only over HTTP — the boundary is real, not cosmetic. Both services share one PostgreSQL instance in different schemas, for cost.

Requirements drift — a BA finding

The existing `docs/requirements/user-stories.md` is strong: 10 epics, Given/When/Then throughout, inline assumptions, and five consolidated open questions. But **three of those five have since been answered in code without the document being updated.**

OPEN QUESTION	DOC SAYS	REALITY
Q1 — PO visibility of analyst rationale	Assumption	IMPLEMENTED AS ASSUMED
Q2 — Framework configurable per jurisdiction?	Open	RESOLVED — DB-driven <code>risk_category</code> table, not an enum
Q3 — Override needs secondary sign-off?	Open	GENUINELY STILL OPEN
Q4 — Majority / unanimous / chair?	Open	RESOLVED — configurable quorum, conservative rule
Q5 — Who approves config changes?	Open	GENUINELY STILL OPEN — self-service today

Closing this loop is cheap and directly serves the 10% requirement-expansion criterion. A judge reading requirements then code will otherwise see decisions drift.

What kind of testing this project needs

Six test types, each earning its place by reaching something the others structurally cannot.

This is a governance product. The thing under test is not "does the button work" but "can an unreviewed AI output reach a human decision-maker, and can anyone tamper with the record afterwards". That pushes testing down to the data layer and out to the authorization boundary — two places a conventional unit-test suite never visits.

TYPE	PROVES	REACHES	STATUS
Unit xUnit + Moq	Service invariants — scoring formula, quorum rule, finalize gate, AI citation guards	Service layer only	74 TESTS
Integration (DB) Testcontainers	Audit trigger rejects UPDATE/DELETE; <code>CHECK</code> constraints hold; each stored function writes its audit row in the same transaction	PostgreSQL — triggers, constraints, functions	NOT STARTED
API / contract Playwright	HTTP status mapping, <code>OperationResult</code> envelope, seed integrity	HTTP boundary	12 TESTS
Authorization Playwright	Role separation across PO / Analyst / Committee	HTTP boundary	GAP RECORDED
E2E journey Playwright UI	The three-role golden path holds end to end	Browser → DB	NOT STARTED
AI evaluation custom C# harness	Model output quality against fixed datasets and deterministic rubrics	The three AI clients	2 OF 12 EXERCISED

Where each suite can and cannot reach

This is the single most important thing to understand about the strategy, and the reason "just use Playwright for everything" would fail.

LAYER	UNIT	API / PLAYWRIGHT	TESTCONTAINERS	UI JOURNEY
webapp (React)	—	—	—	YES
HTTP / controllers	—	YES	—	YES
Services (business rules)	YES	indirect	indirect	indirect
Dapper repos → stored functions	mocked away	indirect	YES	indirect
DB triggers & CHECK constraints	NEVER	NEVER	YES	NEVER

THE LOAD-BEARING CONSEQUENCE

No HTTP endpoint anywhere in the API attempts `UPDATE audit_event`. So no Playwright test, and no UI test, can *ever* prove the append-only trigger works — the API simply never asks. US-9.2, the flagship governance claim of this entire product, is currently asserted in a SQL comment and proven nowhere. Only a Testcontainers suite opening a real connection can close it.

The human-in-the-loop chain the tests protect

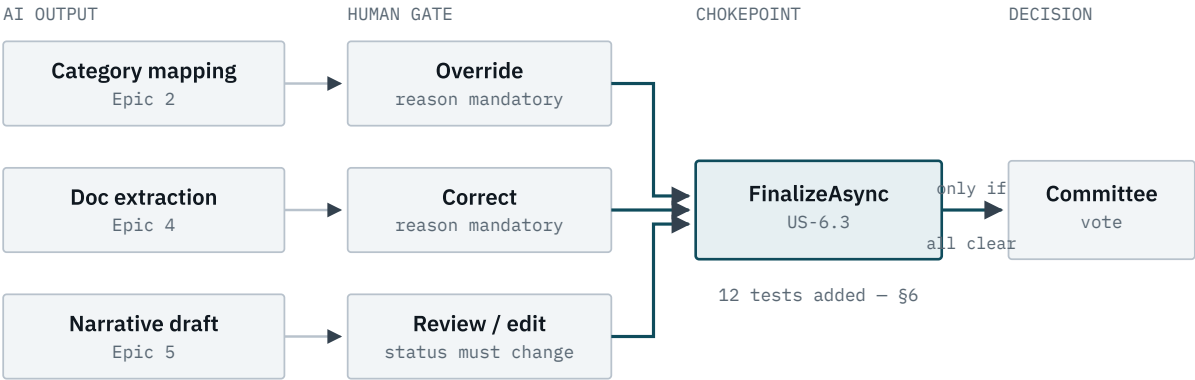


Figure 2. Every AI output converges on one function. `CommitteeService.CastVoteAsync` refuses a vote unless the request is already `PendingCommittee`, so `FinalizeAsync` is the only route in — which is why it was the first thing tested.

BA approach

The requirements baseline exists and is good. The work is closing its own stated gaps.

I am inheriting `user-stories.md`, not writing it. It ends with its own next steps — *"non-functional requirements, a data model / ERD, and a role-based permission matrix"* — and none of the three exist. That list is the BA backlog, and the RBAC matrix in particular has just been validated as urgent by an actual defect (§7, DEF-002).

DELIVERABLE	WHY	PRIORITY
RBAC permission matrix docs/requirements/rbac-matrix.md	3 roles × ~45 endpoints. Writing it exposes real bugs — it already has. Today every controller is a blanket <code>[Authorize]</code> with no role differentiation.	HIGHEST
Non-functional requirements docs/requirements/non-functional-requirements.md	Performance, security, data retention (bank records: 5–7 years), access control, availability. Named explicitly by the existing doc.	HIGH
Traceability matrix docs/requirements/traceability-matrix.md	Every US → implementing code → the test proving it → status. Makes a judge trust everything else, and surfaces which acceptance criteria have no coverage.	HIGH
Data model / ERD docs/requirements/data-model.md	14 schema files, no diagram. Also the natural home for documenting the audit design.	MEDIUM
Update open questions	Mark Q2 and Q4 resolved with links to where; escalate Q3 and Q5 to the team as genuinely open.	QUICK WIN

Working method

- **Acceptance criteria are the spec.** Every test I write names the US/AC it proves, in the test's own doc comment. That is what makes the traceability matrix mechanical to produce rather than a separate invention.
- **Ambiguity becomes a question, not an assumption.** Q3 and Q5 go to the team at standup; they change the design if answered, so they cannot sit in a document unread.
- **Findings carry reproductions.** A defect raised with a failing test is acted on; a defect raised as a doc bullet is discussed. §7 follows that rule.

Testing approach

Risk-based, invariant-led, and deliberately not chasing a coverage percentage.

Prioritisation

With 18 days and one person, test effort follows two things: the judging weights, and what breaks the product's own claims. Those happen to agree.

PRIORITY	WHAT	SERVES
1	Governance invariants — finalize gate, audit immutability, residual > 0, individual votes	15% human-in-the-loop
2	CI running tests + evals at all	20% SDLC automation
3	Adversarial AI eval cases	10% evaluation framework
4	Authorization / RBAC	5% production readiness
5	Three UI journeys	demo evidence

Rules we hold ourselves to

- **A test that cannot fail is not a test.** Every governance test is verified by mutation — break the rule deliberately, confirm the test catches it, revert. Done for the finalize gate already (§6).
- **Known gaps are recorded, not hidden.** Where the product does not yet do the right thing, the test asserts the *right* behaviour and is marked expected-to-fail, so the fix announces itself. It is never deleted or weakened to keep the board green.
- **Characterization over silence.** Where current behaviour is arguably wrong but changing it is the dev's call, we pin it with a clearly-commented test so any future change is deliberate.
- **Never let a dead environment look like a pass.** Expected-to-fail tests pass when they fail for *any* reason, including a refused connection — so the suite aborts up front if the API is unreachable.
- **Scope claims honestly.** If a probe used a non-existent id, the report says so and says what it therefore does *not* prove.

Defect workflow

1. Reproduce with a command or a test — no defect without one.
2. Classify severity by what it breaks in the *product's own claims*, not by crash loudness.
3. Land the failing/characterization test on the QA branch first.

4. Raise at standup with the repro and the owning user story.
5. When fixed, the marker flips and tells us — remove the annotation, not the test.

Entry and exit criteria

GATE	CRITERIA
Entry to test	Stack starts clean from a fresh clone; schema + seed apply; <code>/api/Ping</code> returns 200; the user story has acceptance criteria.
Exit for submission	All four governance invariants proven by a test that has been shown to fail when broken; CI runs unit + API suites on every PR; all 12 eval cases exercised against a real model; every open defect either fixed or documented with a repro.

Automation strategy

Four tools, each chosen for the layer it can reach — and one deliberate decision about what not to automate.

TOOL	LAYER	WHY THIS AND NOT SOMETHING ELSE
xUnit + Moq	Services	Already the repo's convention. No I/O, milliseconds, runs anywhere. Every dependency mocked through its existing interface.
Playwright	HTTP + browser	One tool for API and UI. Auth0 is bypassed in local dev, so there is <i>no login flow to automate</i> — normally the most painful part of E2E setup.
Testcontainers	PostgreSQL	The only way to reach triggers and CHECK constraints. Docker is already a prerequisite, so this adds no new dependency.
Custom C# eval harness	AI clients	Already built. Scores the <i>production</i> AI clients, not reimplementations, with deterministic rubrics where the domain allows.

ENGINEERING JUDGEMENT — WHERE WE DELIBERATELY DO NOT AUTOMATE

The UI suite stays at three journeys, not thirty. UI tests are the most expensive evidence per hour in this repo — they need the Vite dev server plus both .NET services plus Postgres — and a broad UI suite would mostly re-prove what 74 unit tests already cover far more cheaply. Three role journeys (PO submits → Analyst assesses and finalizes → Committee votes) are the product demo and exercise the whole governance chain. That is the right amount.

The CI gap — priority one

Two Azure DevOps pipelines exist and both are path-filtered to `iac/environments/dev/*`. **Nothing in CI ever runs `dotnet test` or the eval suite.** On the branches page, the application branches show a blank check-status column while every IaC branch shows a green tick — that is the gap made visible.

Proposed pipelines

PIPELINE	TRIGGER	STEPS
app-pr-validation.yml	PR touching <code>src/**</code> , <code>tests/**</code> , <code>webapp/**</code>	<code>dotnet build</code> → <code>dotnet test</code> → publish TRX → Playwright API suite against a service container
evals-nightly.yml	Scheduled + manual	<code>dotnet run</code> the eval harness with the Foundry key from a variable group; publish the Markdown report as an artifact

Mirroring the existing IaC pipeline's structure — including its Claude review step, which already works — makes this read as deliberate rather than bolted on, and evidences SDLC automation breadth across stages.

A note on the IaC pipelines

Trivy, Checkov and `terraform plan` are all currently **soft-fail**. A PR with real security findings still goes green and auto-applies to `main`. On a project whose entire thesis is governance, that is worth raising with DevOps — it is a two-line change.

06

Work completed so far

Three commits on `qa/ba-qa-workstream`, all verified against a running stack.



Environment brought up

The local stack now runs end to end, which no prior work had established. Note the setup script assumes `psql` on the host — it is not installed on this machine, so both schemas were applied through the container instead. Worth fixing in `ops/setup-local-db.sh` so a fresh clone works for everyone.

COMPONENT	STATUS
PostgreSQL + Azurite containers	HEALTHY :5433 / :10000
Both schemas + seed applied	6 USERS / 4 ROLES, 4 FFIEC CATEGORIES
Workbench API	:5210 RESPONDING
.NET 10.0.401 SDK	MATCHES NET10.0 TARGET

Commit 1 – the finalization gate 1ca7a37

`AssessmentService.FinalizeAsync` is the single chokepoint in Figure 2, and it **had no test at all**. Added 12, taking the suite from 50 to 62.

TEST	PROVES
BlocksAndNamesTheCategoryWhenItsNarrativeIsStillAiDrafted	US-5.1 AC2
BlocksWhenAMappedCategoryHasNoNarrativeSectionAtAll	US-6.3 AC1
BlocksWhenAMappedCategoryHasNoPolicyRelianceDecision	US-3.2 AC2
NeverFinalizesTheAssessmentWhenBlocked	the governance assertion
ListsEveryOutstandingReasonNotJustTheFirst	US-6.3 AC1 — "lists what's outstanding"
RecordsTheFinalizingAnalystWhenEveryGateIsSatisfied	US-6.3 AC3 — not anonymous
TreatsAReviewedOrEditedSectionAsSatisfied	US-6.3 AC2 (×2 cases)
IgnoresACategoryTheAnalystDeactivated	Epic 2 override
AcceptsAnAssessmentWideRelianceForEveryMappedCategory	null-category reliance branch
ReportsReadyWhenNoCategoriesAreMappedAtAll	characterization — see DEF-004

Verified by mutation

Passing tests prove nothing on their own. The gate condition was temporarily replaced with `if (false)` — effectively deleting it — and the suite re-run:

Failed: 5, Passed: 57, Total: 62

Exactly the five `FinalizeAsync` gate tests failed; the seven readiness and delegation tests correctly stayed green because they never pass through the gate. That is the right kill pattern. Production code was then restored byte-for-byte (`git diff -- src/` empty) and the suite returned to 62/62.

Commits 2 & 3 — Playwright API suite `da40d82`, `2f2f8b7`

Scaffolded `tests/e2e` and ran it against the live stack: **12 passed** — 6 genuine assertions plus 6 expected-to-fail markers recording DEF-001 through DEF-003.

The suite includes a smoke spec that pins the `OperationResult` envelope, the six seeded users across four roles, and the four FFIEC category codes — so a broken stack or a partial seed is always distinguishable from a real finding. It also keeps two *passing* validation tests (a mitigation factor of `1.0` is correctly refused), which keeps the authorization finding precise rather than overstated into "the API accepts anything".

AI evaluation baseline

The harness runs and behaves exactly as documented: **2 pass, 0 fail, 10 skip** without a key. The two that pass are deterministic guard-clause cases that never call a model. Everything else reports `Skip` rather than `Fail`

— the right distinction, since "we didn't check" is not the same claim as "the model got it wrong".

OPEN ITEM

The Foundry project is now provisioned (`gpt-4.1-mini` , `proj-gh.services.ai.azure.com`) and that host matches the route the code already calls. Setting the four `FOUNDRY_*` environment variables moves 10 skipped cases into real pass/fail. **Expect some genuine failures** — these prompts and rubrics were written against Claude, and gpt-4.1-mini is a smaller model. That is not a problem to paper over; measuring a model swap is exactly what an evaluation framework is for, and "here is what moved" is a far stronger story than twelve green ticks.

Defects & reproduction steps

Four findings, each with a command you can run in front of the team.

All four are on synthetic data, in a Development configuration, with no Auth0 tenant and no production deployment. **None is a live breach.** They are raised because each one contradicts a claim the product makes about itself.

DEF-001 **HIGH** SECURITY

The full user directory is readable with no credential at all

`GET /api/User` returns every user's display name, email address and role to a caller presenting no `Authorization` header whatsoever.

`DevBypassAuthHandler` authenticates *every* request as a generic `system-dev` principal whenever `AUTH0_DOMAIN` is blank in Development. So `[Authorize]`, which every controller carries, currently proves nothing at all.

Reproduce

```
curl -i http://localhost:5210/api/User
```

Expected 401 · **Actual** 200 with all six seeded users.

BREAKS

Access control; the principle that role separation is enforced

TEST

authorization.spec.js → "the user directory is not readable without a credential"

OWNER

Suleman (API / auth)

No endpoint enforces role — the acting user is self-declared

Every write endpoint reads the acting user from the **request body**, not from the token — `CastCommitteeVoteInput.CommitteeMemberUserId`, `FinalizeBody.ActorUserId`, `UpsertScoringConfigInput.ActorUserId`. Nothing checks that the declared actor holds the role the action requires. Roles are seeded and never read.

Reproduce

```
# Resolve the Product Owner's id
curl -s http://localhost:5210/api/User

# Cast a committee vote as that Product Owner
curl -i -X POST http://localhost:5210/api/Committee/Vote \
  -H 'Content-Type: application/json' \
  -d '{"assessmentId":"<id>","committeeMemberUserId":"<product-owner-id>","vote":"Appr
```

PROBE (ACTOR = PRODUCT OWNER)	EXPECTED	ACTUAL
POST /api/Committee/Vote	403	400
POST /api/Assessment/{id}/Finalize	403	500
POST /api/Scoring/Config	403	500

No endpoint returned 403. The role was never consulted on any of them.

SCOPE THIS HONESTLY WHEN PRESENTING

These three probes used a deliberately non-existent id, so each was rejected on *data* grounds before role could have mattered. They prove the role check does not run first. They do **not** prove a request against a valid, committee-routed assessment would be accepted — confirming that needs a seeded `PendingCommittee` assessment, which is follow-up work. DEF-001 carries no such caveat and is the stronger demo.

BREAKS

US-8.2 AC5, US-6.3 AC3, US-10.1

NOTE

Configuring Auth0 will *not* fix this — the authorization layer Auth0 would feed does not exist

OWNER

Suleman (API / auth)

DEF-003

MEDIUM

ROBUSTNESS

An unknown id surfaces as HTTP 500

A well-formed but non-existent id reaches Postgres and the resulting exception escapes as a 500 rather than a 4xx. `BaseApiController.ExecuteAsync` catches and logs it, but its fallback is `OperationResult.Failure` → 500 — so "you sent an id that does not exist" reads to the caller as "the server broke". `OperationResult` already has `NotFound` and `BadRequest` shapes built for exactly this.

Reproduce

```
curl -i -X POST http://localhost:5210/api/Scoring/Config \  
-H 'Content-Type: application/json' \  
-d '{"riskCategoryId":"00000000-0000-0000-0000-0000000000ff",  
    "maxMitigationFactor":0.5,"reason":"probe","actorUserId":"<analyst-id>"}'
```

Server log confirms the cause:

```
Npgsql.PostgresException 23503: insert or update on table "scoring_config"  
violates foreign key constraint "scoring_config_risk_category_id_fkey"
```

BREAKS

API contract; masks client errors as server faults

TEST

authorization.spec.js → "Error handling" block (2 tests)

OWNER

Suleman (API)

DEF-004

MEDIUM

GOVERNANCE

An assessment with no mapped categories finalizes vacuously

With zero active risk categories, both readiness loops in `CheckReadinessAsync` iterate over an empty list, so `IsReady` is **vacuously true**. `FinalizeAsync` then succeeds and routes the assessment to committee with no narrative and no policy review whatsoever.

Reachable two ways: the AI proposes nothing — which is US-2.1 AC3's *legitimate* "no framework mapping configured" path, correctly returning empty rather than guessing — or an analyst deactivates every proposed category.

Reproduce

Covered by the characterization test

`CheckReadinessAsync_ReportsReadyWhenNoCategoriesAreMappedAtAll`. To show it live: open a workspace on a change request, propose no categories, and call Finalize — it succeeds.

BREAKS

"Don't let an AI output reach the committee stage without a review gate"

FIX

One guard in `CheckReadinessAsync`: zero active categories is not ready

STATUS

Pinned as characterization, deliberately not "fixed" by QA — it is the dev's call

OWNER

Suleman (Assessment module)

How to showcase this to the team

A five-minute live demo, in this order. It builds from the most unambiguous finding to the most nuanced, which is also least-to-most likely to attract debate.

1. **Start the stack** — containers plus the API, roughly 30 seconds.
2. **Run** `curl http://localhost:5210/api/User` **with no header**. Six users, names and emails, on screen. This needs no explanation and no caveat.
3. **Run the Playwright suite**. 12 passed in ~4 seconds. Then open the HTML report and show the six expected-to-fail markers with their recorded actual status codes — the gaps are *in* the green run, not hidden by it.
4. **Show the mutation result**. Break the finalize gate to `if (false)`, re-run, 5 fail, revert. This is the strongest single thing to show: it proves the tests are real, which is the question any reviewer should ask about a green board.
5. **Close with DEF-002's caveat, out loud**. Saying plainly what the probes do not prove is what makes the rest of the report credible.

ASKS FOR THE TEAM

Suleman — DEF-001 to DEF-004; and a decision on open questions Q3 (does a scoring override need secondary sign-off?) and Q5 (who approves configuration changes?).

Suraj — merge `release/1.00` into `main` so application CI becomes possible at all; and consider hard-failing Trivy/Checkov on the IaC pipelines.

Everyone — API keys are being shared in Google Chat on a project whose own ops plan specifies Key Vault. Worth rotating before submission.

Prepared from a full read of both branches — `main` (IaC) and `release/1.00` (the application, 262 files). All test results in §6 and all status codes in §7 were observed against a running local stack on 12 September 2026, not inferred. Where a claim is unverified, it is marked as such. · Live version: `claude.ai/code/artifact/0181e636-fb00-4065-9cc1-dd402040ae7c`